

Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Jane Open Access   

Dummy University Computing Laboratory, [optional: Address], Country

My second affiliation, Country

Joan R. Public¹  

Department of Informatics, Dummy College, [optional: Address], Country

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent convallis orci arcu, eu mollis dolor. Aliquam eleifend suscipit lacinia. Maecenas quam mi, porta ut lacinia sed, convallis ac dui. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Suspendisse potenti.

2012 ACM Subject Classification Replace ccsdesc macro with valid one

Keywords and phrases Dummy keyword

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Funding *Jane Open Access:* (Optional) author-specific funding acknowledgements

Joan R. Public: [funding]

Acknowledgements I want to thank ...

1 Introduction

ELPI is a dialect of λ PROLOG (see [14, 15, 7, 12]) used as an extension language for the ROCQ prover (formerly the COQ proof assistant). ELPI has become an important infrastructure component: several projects and libraries depend on it [13, 3, 4, 19, 8, 9]. Examples include the Hierarchy-Builder library-structuring tool [5] and Derive [17, 18, 11], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, ELPI gained a static analysis for determinacy [10] to help users tame backtracking. ROCQ users are familiar with functional programming but not necessarily with logic programming and uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checkers identifies predicates that behave like functions, i.e., predicates that commit to their first solution and leave no *choice points* (places where backtracking could resume).

This paper reports our first steps towards a mechanization, in the ROCQ prover, of the determinacy analysis from [10]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantic depart from a pure logical reading.

We formalize two operational semantics for PROLOG with cut. The first is a stack-based semantics that closely models ELPI's implementation and is similar to the semantics mechanized by Pusch in ISABELLE/HOL [16] and to the model of Debray and Mishra [6, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations used by standard PROLOG abstract machines [20, 1], but it makes reasoning about the scope of *cut* difficult. To address that limitation we introduce a tree-based semantics in which the branches pruned by *cut* are explicit and we prove the two semantics equivalent. Using the

¹ Optional footnote, e.g. to mark corresponding author



```

Inductive P := IP of nat. Inductive D := ID of nat. Inductive V := IV of nat.

Inductive Tm :=
  | Tm_P of P
  | Tm_D of D
  | Tm_V of V
  | Tm_App of Tm & Tm.

Record Unif := {
  unify : Tm → Tm → Σ → option Σ;
  matching : Tm → Tm → Σ → option Σ;
}.

```

■ Figure 1 Tm types

■ Figure 2 Unif type

tree-based semantics we then show that if every rule of a predicate passes the determinacy analysis, the call to a deterministic predicate does not leave any choice points.

2 Common code: the language

Before going to the two semantics, we show the piece of data structure that are shared by the them. The smallest unit of code that we can use in the language is an atom. The atom inductive (see Type 1) is either a cut or a call. A call carries a term (see Figure 1). A term (Tm) is either a predicate, a datum, a variable or the binary application of two terms.

```

Inductive A := cut | call : Tm → A. (1)
Record R := mkR { head : Tm; premises : list A }. (2)
Record P := { rules : seq R; sig : sigT }. (3)
Definition Σ := {fmap V → Tm}. (4)
Definition bc : P →  $\mathcal{F}$  → Tm → Σ →  $\mathcal{F}$  * seq (Σ * seq A) := (5)

```

A rule (see Type 2) has a head of type term and a list of premises, that are a list of atoms. A program (see Type 3) is made by its rules and the signatures of predicates, **sigT** is a mapping from predicates to their signatures, signatures represent the definition of type from the simply typed lambda calculus, i.e. it is either a base type or an arrow for term application. We decorate arrows to know the mode (input or output) of the application argument.

A substitution (see Type 4) is a mapping from variables to terms. It is the output of a successful query. The empty substitution is noted ϵ .

The backchain function (bc, see Type 5) takes: a program P ; a set fv of variables ($\mathcal{F}_V$); a query q ; and the substitution σ . A backchain is performed when the current atom is a **call**. In this case, it start by recursively *dereferencing* the variables in σ , i.e. replacing all the variables in the query by their term mapping in σ .

TODO: free variables and program freshness

Then it takes the head of the term, which should be rigid, i.e. a predicate p , and look for the signature s of p in the program. Then, for each rule r in P , it unifies the head of r with q . Unification is possible thanks to the unificator U , whose type is shown in Figure 2. U explains how to unify terms up to standard unification (for output terms) or matching (for input terms). The function *unif* (resp. *matching*) unifies (resp. matches) the two terms under σ , it return an extension of σ if the two terms can be unified (resp. matched), and None otherwise. The result of a backchain is made by filtering the rules in P that can be used on the given query. In particular, for each rule filtered rules r , it returns a pair containing the extension of σ making the head of r and q unify and the list of premises of r .

```

f 1 2.   f 2 3.   r 0 0.   r 0 1.   r 0 2.
g X Z :- r X Y, f Y Z, !. % r1
g X Z :- f X Y, f Y Z.    % r2

```

Inductive tree :=
 | KO | OK | TA of A
 | Or of option tree & Σ & tree
 | And of tree & seq A & tree.

■ Figure 3 Small ELPI program example

■ Figure 4 Tree inductive

2.1 The cut operator

ELPI is a dialect of λ PROLOG with the hard cut operator and, in the language we have shown before, *cut* is an atom. The semantics of the cut operator adopted in the ELPI language corresponds to the *hard cut* operator of standard SWI-PROLOG. This operator has two primary purposes. First, it eliminates all alternatives that are created either simultaneously with, or after, the introduction of the cut into the execution state.

The ELPI program we will use as an example in the following sections is shown in Figure 3.

To illustrate the high-level description of *cut*, consider the program P shown in Figure 3 and the query $q = g \ 0 \ R$. Both rules for g can be used on the query q . Backchaining q in P from the empty (ϵ) substitution returns the list $[(\sigma_1, [r \ X \ Y, f \ Y \ Z, !]), (\sigma_2, [f \ X \ Y, f \ Y \ Z])]$ with σ_1 and σ_2 both equal to $\{X \mapsto 0, Z \mapsto R\}$. The two elements of the list are generated respectively from the rules r_1 and r_2 .

The next step of the interpretation corresponds to the backchaining of $r \ X \ Y$ from σ_1 . This returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [])]$ with $\sigma_3 := \sigma_1 + \{Y \mapsto 0\}$, $\sigma_4 := \sigma_1 + \{Y \mapsto 1\}$ and $\sigma_5 := \sigma_1 + \{Y \mapsto 2\}$. The three lists of premises are empty since all rules for r have no premises.

Backchaining $f \ Y \ Z$ in σ_3 gives no solution, therefore, by backtracking, we backchain $f \ Y \ Z$ in σ_4 . This gives a solution, namely $\sigma_6 := \sigma_4 + \{R \mapsto 2\}$.

We now reach the cut operator and can see its double side behavior. In the current execution state, we have still two unexplored alternatives: we can backtrack on the third solution for r that assign Y to 2 and on the rules r_2 . We can note, however, the first alternative have been generated after the cut introduction, and the second alternative have been generated at the same moment of the cut. Therefore both are cut away leaving no choice point. The final complete solution is: $\{X \mapsto 0, Z \mapsto R, Y \mapsto 1, R \mapsto 2\}$, where X , Y and Z can be ignored since they are the result internal unification.

3 Semantics intro

We propose two operational semantics for a logic program with cut. The two semantics are based on different syntaxes, the first syntax (called *tree*) exploits a tree-like structure and is ideal both to have a graphical view of its evolution while the tree is being interpreted and to prove lemmas over it. The second syntax, called *elpi*, is the ELPI's syntax and has the advantage of reducing the computational cost of cutting and backtracking alternatives by using shared pointers. We aim to prove the equivalence of the two semantics together with some interesting lemmas of the cut behavior.

4 Tree semantics

The tree state The tree inductive, shown in Figure 4. We distinguish 5 main cases: KO , OK , and are special meta-symbols representing, respectively, the basic failed and a successful

se metti $r1 = g \ A$
 $B :- f \ A \ B$. allora
 $g \ e \ f$ sono fun-
 zioni, e puoi spie-
 gare anche l'idea
 del detcheck qui

```

Fixpoint get_end s A :  $\Sigma$  * tree :=
  match A with
  | TA _ | KO | OK  $\Rightarrow$  (s, A)
  | Or None s1 B  $\Rightarrow$  get_end s1 B
  | Or (Some A) _ _  $\Rightarrow$  get_end s A
  | And A _ B  $\Rightarrow$ 
    let (s', pA) := get_end s A in
    if pA == OK then get_end s' B
    else (s', pA)
  end.

Definition get_subst s A := (get_end s A).1.
Definition path_end A := (get_end  $\epsilon$  A).2.
Definition success A := path_end A == OK.
Definition failed A := path_end A == KO.
Definition path_atom A :=
  if path_end A is TA _ then true
  else false.

```

(a) Definition of *get_end*(b) Functions from *get_end*

■ **Figure 5** *get_end* and derives functions

110 trees. These symbols are considered meta because they are internal intermediate symbols
 111 used to give structure to the tree.

112 The *TA* constructor (acronym for tree-atom) is the constructor of atoms in the tree. They
 113 are either the cut operator or a call, in these cases, the interpreter should evaluate the atom
 114 by either cutting the subtrees in the scope of the cut or backchaining the call in the program.

115 The two recursive cases of a tree are the *Or* and *And* nodes. The *Or* node $A \vee B_\sigma$ denotes
 116 a disjunction between two trees *A* and *B*. The first branch is optional, if absent it represents
 117 a dead tree, i.e. a tree that has been entirely explored. The second branch is annotated
 118 with a suspended substitution σ so that, upon backtracking to *B*, σ is used as the initial
 119 substitution for the execution of *B*.

120 The *And* node $A \wedge_{B_0} B$ represents a conjunction of two trees *A* and *B*. We call B_0 the
 121 reset point for *B*; it is used to restore the state of *B* to its initial form if a backtracking
 122 operation occurs on *A*. An example of this behavior is shown in Section 4.2

123 A graphical representation of a tree is shown in Figure 6a. To make the graph more
 124 compact, the *And* and *Or* nodes are n-ary rather than binary, and the children of these
 125 nodes associate to the right. The *KO* node acts as the neutral elements in the *Or* list, while
 126 *OK* is the neutral element of the *And* list.

127 *The tree interpreter* The interpretation of a tree is performed by two main routines: *step*
 128 and *next_alt* that traverse the tree depth-first, left-to-right. Then, then *runT* inductive
 129 makes the transitive closure of *step* and *next_alt*: it iterates the calls to its these
 130 auxiliary functions as much as needed. In Types 7–9 we give the types contrats of these
 131 symbols where $\mathcal{F}_v$ is a set of variable names.

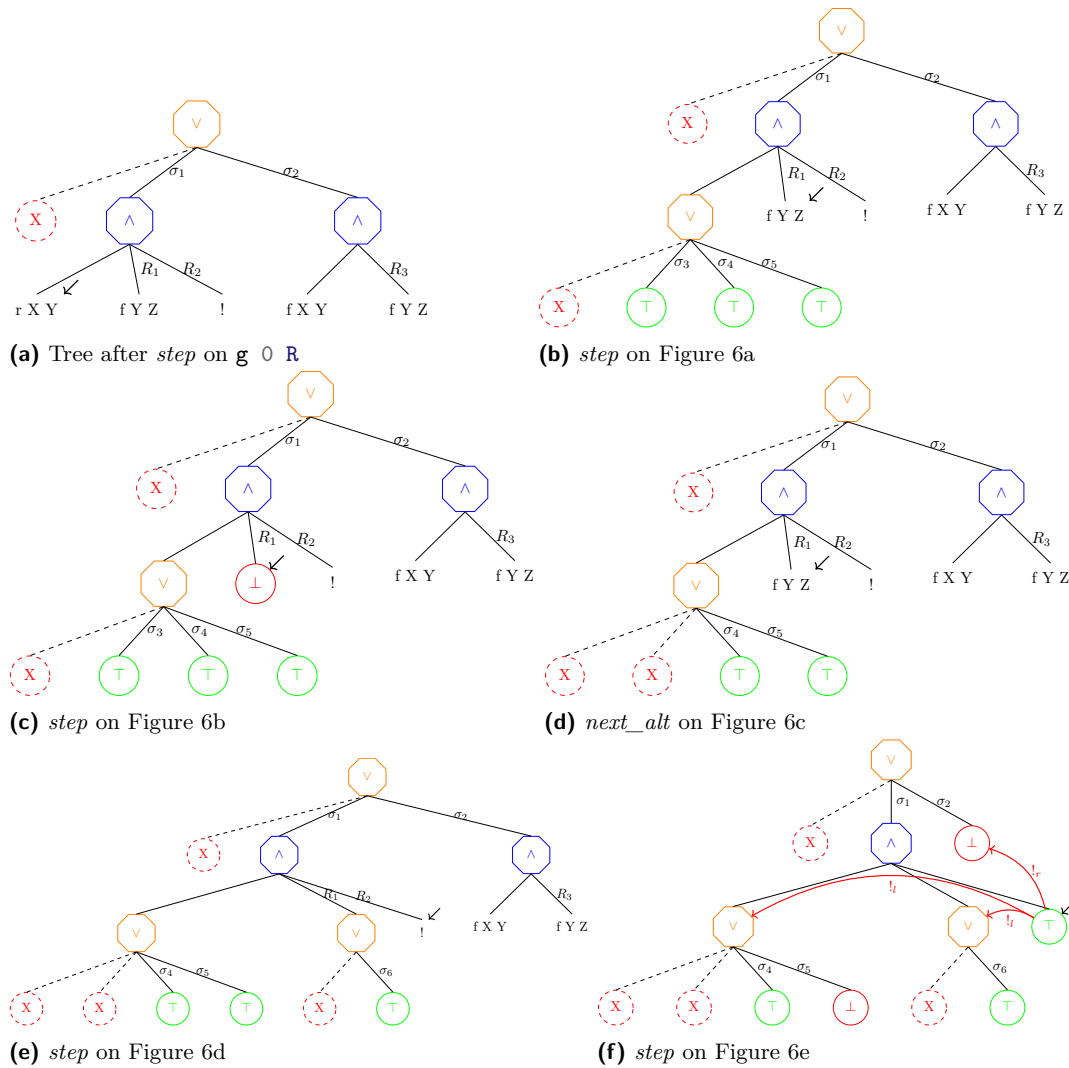
132 **Inductive** tag := Expanded | CutBrothers | Failed | Success. (6)

133 **Definition** step : $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \Sigma \rightarrow \text{tree} \rightarrow (\mathcal{F}_v * \text{tag} * \text{tree}) :=$ (7)

134 **Definition** next_alt : $\mathbb{B} \rightarrow \text{tree} \rightarrow \text{option tree} :=$ (8)

135 **Inductive** runT (p : $\mathbb{P}$) : $\mathcal{F}_v \rightarrow \Sigma \rightarrow \text{tree} \rightarrow$
 $\Sigma \rightarrow \text{option tree} \rightarrow \text{Prop} :=$ (9)

136 The tree interpreter, as in prolog, explores the tree in DFS strategy, to discover the
 137 substitution and the leaf of the tree that should be interpreted. These two pieces of
 138 information can be retrieved for a substitution *s* and a tree *A* thanks to the *get_end* routine,
 139 shown in Figure 5a If the tree is already a leaf, *get_end* returns its inputs. Otherwise, if the
 140 tree is a disjunction, the path continues on the left subtree, if it exists, otherwise *get_end*
 141 contues on the ths using the substitution stored in the *Or* node. In the case of a conjunction,



■ **Figure 6** Some tree representations²

142 if *get_end* of the lhs returns the tree *OK*, then the *get_end* continues in the rhs, otherwise
 143 we return the result of the lhs. We derive some useful definition from *get_end*. They are
 144 listed in Figure 5b.

145 We have pointed by a black arrow the *path_end* of all the trees in Figure 6.

146 4.1 The *step* procedure

147 The *step* procedure takes as input a program, a set of variables, a substitution, and a tree,
 148 and returns an updated set of variables, a *tag*, and an updated tree.

149 The set of variables v given in input to *step* are used and updated when a backchaining
 150 operation takes place. A call to *step* produces a good result if v contains all the variables

² The substitution $\sigma_1 \dots \sigma_6$ are from Section 2.1, R_1, R_2 and R_3 are the reset points and correspond respectively to the list of atoms $[f \ Y \ Z, !], [!]$ and $[f \ Y \ Z]$

not true in figure:fix + also fix in Section 2.1

151 in the tree. This guarantees that any call to *bc* returns a list of atoms whose variables are disjoint from the variables of the current working tree.

152 The *tag* (see Type 6) indicates the kind of an internal tree step: **CutBrothers** denotes the interpretation of a superficial cut, i.e., a cut whose parent nodes are all *And*-nodes. **Expanded** denotes the interpretation of non-superficial cuts or predicate calls. **Failure** and **Success** are returned for, respectively, *failed* and *success* trees.

157 The step procedure is intended to evaluate atoms, that is, it leaves unchanged the tree iff its *path_end* is not an atom, i.e. if it is a success- or a failed-tree.

159 **Lemma** *succ_step_iff* $u\ p\ v\ s\ A$: $\text{success } A \leftrightarrow \text{step } u\ p\ v\ s\ A = (v, \text{Success}, A)$. (1)

160 **Lemma** *fail_step_iff* $u\ p\ v\ s\ A$: $\text{failed } A \leftrightarrow \text{step } u\ p\ v\ s\ A = (v, \text{Failed}, A)$. (2)

161 The interpretation of a call to a term *c* starts by calling the *bc* function on *c*. The result of the backchain is a list *l*. If *l* is empty then the current call is replaced by the *KO* node, otherwise it is replaced by a right skewed tree made by *Or* inner-nodes and each leave correspond to a list of atom *e* in *l*. If *e* is empty then it is mapped to the *OK* tree, otherwise it is mapped to a right-skewed tree made by *And* inner-nodes.

sottolineare sostituzioni

166 In Figure 6a, we provide a clear example of a call to *step* on the query $q\ 0\ R$ on the program *P* in Figure 3. Let *c*₀, *c*₁ and *c*₂ be the children of the root. *c*₀, by construction is the *None* tree, while *c*₁ and *c*₂ correspond respectively to the premises of rules *r*₁ and *r*₂ in *P*. We need the first *None* subtree so that we can attach a substitution to *c*₁. In particular, we note that the *c*₁ (resp. *c*₂) are related to the substitution σ_1 (resp. σ_2). The value of σ_1 and σ_2 are the same as in Section 2.1. The reset points in the tree are marked with the *R* symbol. $R_1 := [f\ Y\ Z, !]$, $R_2 := [!]$ and $R_3 := f\ Y\ Z$, they are essentially the list of atoms that allowed to generate the current subtree. Other example of call to the *step* procedure triggering a backchain are Figures 6b, 6c, and 6e.

175 The interpretation of a cut has three main impacts: at first the cut is replaced by the *OK* node, then some special subtrees, in the scope of the *Cut*, are cut away: in particular we need to soft-kill the left-siblings of the *Cut* and hard-kill the right-uncles of the the *Cut*.

178 ► **Definition 1** (Left-siblings (resp. right-sibling) and Right-uncles). Given a node *A*, the left-siblings (resp. right-sibling) of *A* are the list of subtrees sharing the same parent of *A* and that appear on its left (resp. right). The right-uncles of *A* are the list of right-sibling of the father of *A*.

182 ► **Definition 2** (Hard-kill, $!_r$ and Soft-kill, $!_l$). Given a tree *t*, hard-kill replaces the given subtree with the *KO* node. If *t* is a success-tree, then soft-kill replaces with *KO* all subtrees that are not part of the path in *t* leading to the *OK* node.

185 An example of the impact of the cut is show in Figure 6f. In Figure 6e, we see that the *path_end* is a cut. The cut is replaced by the *OK* node and it need to cut-away some alternatives in the tree. The red arrows indicate the scope of the cut, they are labeled with $!_r$ and $!_l$ to explain which kind of cut has been performed on each pointed subtree. We note that, after the evaluation of a cut, the path leading to the cut node is not affected by $!_l$, the success under the substitution σ_4 is kept, while the success under the substitution σ_5 is replaced by the *KO* node. If we refer to the paragraph in Section 2.1, we are discarding the third rule of the *r* predicate.

193 4.2 The next_alt procedure

194 The *step* procedure alone is not sufficient to reproduce entirely the behavior of the full expected prolog interpreter. As proved in Lemma 1 and Lemma 2, *step* does not make any

progression on the tree being interpreted. To solve this problem, we need a routine capable of backtracking in case of failures. This routine return the backtracked tree if it exists, *None* otherwise.

In Figure 6c, we are facing a failure: the interpreter has evaluated the atom $\mathbf{f} \ X \ Y$ under the substitution σ_3 which assigns Y to 0, but, in Figure 3, we have no rule satisfying this query. The role of *next_alt*, in this example, is to kill the path leading to this failure, while resetting the failure to its original shape. We see how the reset point plays a major role in this case. At first we are killing the *OK* node under the substitution σ_3 , since this tree have been entirely explored and led to no solution, then we replace the *KO* node with the information stored in R_1 . That is, thanks to R_1 , we are restoring the $\mathbf{f} \ Y \ Z$ call, so that we are able to evaluate this node under the substitution σ_4 . More concretely, we have uncessesfully tried rule $\mathbf{r} \ 0 \ 0$, therefore, we need to continue by trying rule $\mathbf{r} \ 0 \ 1$.

Furthermore, *next_alt* accomplishes to another important task. Its behavior changes depending to the value of the boolean it receives in entry: *next_alt false A* aims to recursively cancel all the failures (if any) in A , whereas, *next_alt true A* aims to cancel the first success in A . This second behavior is helpful since, we need to progress the tree in case of success. For example, consider the tree in Figure 6f, it contains a success, mapping namely $\mathbf{R}$ to 2. The interpreter should return this success, but, due to the “non-deterministic” behavior of the prolog system, we also need to return all the other solution that could be available by evaluating the rest of the tree. After the replacement of the *path_end* in Figure 6f with *KO*, we remain with a tree having all failing path. The *next_alt* procedure keeps removing them until discovering that the tree have no worth-to explore paths, therefore it will return *None*.

Some interesting property of *next_alt* are shown below and allow to see how *next_alt* complements *step* wrt failed, success and *path_atom* trees.

Lemma *path_atom_next_alt_id* $\mathbf{b} \ A$: $\text{path_atom } A \rightarrow \text{next_alt } \mathbf{b} \ A = \text{Some } A$. (3)

Lemma *next_alt_failedF* $\mathbf{b} \ A \ A'$: $\text{next_alt } \mathbf{b} \ A = \text{Some } A' \rightarrow \text{failed } A' = \text{false}$. (4)

4.3 The *runT* inductive

The inductive *runT* is modeled as a function: it takes as input a program, a set of variables, an initial substitution σ_0 , and a tree t_0 , and returns a substitution σ_1 together with an optional updated tree t_1 . The substitution σ_1 represents the most-general unificator that makes the execution of the tree t_0 succeed starting from the initial substitution σ_0 , σ_1 is an extension of σ_0 . The tree t_1 is the updated tree containing the alternatives that have not yet been explored. If the tree contains no solution, then *None* is returned.

The procedure *runT* is based on three main derivation rules, shown in Figure 7. If the *path_end* of the tree t is a success, the substitution is retrived in t from σ_1 and the input tree is cleaned of its successful path. If the *path_end* of the tree is an atom, then *step* is invoked to evaluate this atom, and *runT* is recursively called on the new tree. Finally, if the *path_end* of the tree is a failure, *next_alt* is called to clear the failed path; if the resulting cleaned tree exists, *runT* is recursively called on it.

TODO: start

DIRE CHE RUN TERMINA

In our implementation, *runT* gives the garantee that the program interpretation terminates, since it always return a substitution, but we can easily make it also accept diverging program by making *runT* return an optional substitution

TODO: end

$$\begin{array}{c}
\frac{\text{success } A \quad \text{get_subst } s_0 \ A = s_1 \quad \text{next_alt true } A = B}{\text{runT } v_0 \ s_0 \ A \ s_1 \ B} \text{ RUN_DONE} \\
\\
\frac{\text{path_atom } A \quad \text{step u p } v_0 \ s_0 \ A = (v_1, \text{st}, B) \quad \text{runT } v_1 \ s_0 \ B \ s_1 \ r}{\text{runT } v_0 \ s_0 \ A \ s_1 \ r} \text{ RUN_STEP} \\
\\
\frac{\text{failed } A \quad \text{next_alt false } A = \text{Some } B \quad \text{runT } v_0 \ s_0 \ B \ s_1 \ r}{\text{runT } v_0 \ s_0 \ A \ s_1 \ r} \text{ RUN_FAIL}
\end{array}$$

■ **Figure 7** Rule system for *runT*

241 5 Elpi semantics

242 We now want to introduce the elpi semantics. The interpreter we show reflects the interpreter
 243 of the ELPI language and is an operational semantics close to the one picked by Pusch in
 244 [16], in turn closely related to the one given by Debray and Mishra in [6, Section 4.3]. Pusch
 245 mechanized the semantics in Isabelle/HOL together with some optimizations that are present
 246 in the Warren Abstract Machine [20, 1].

247 The inductive representing a state of the ELPI language is shown below.

```

Inductive alts := no_alt | more_alt of (Σ * goals) & alts
with goals := no_goals | more_goals of (A * alts) & goals .

```

248 An elpi state is an enhanced two-dimension list. The outermost list represents the list
 249 of alternatives in disjunction accompagnate with the substitution that should be used to
 250 for their interpretation. The innermost list is a list of atom, representing a list of goals in
 251 conjunctions. These goals are decorated with a pointer to an elpi state, and are used to keep
 252 trace of the alternatives that should be kept when a cut is interpreted. We call these, special,
 253 alternatives the cut-to alternatives.

254 The idea of the ELPI interpreter is to receive a list of alternatives. The first alternative
 255 consists of a list of goals. Four cases must be taken into account; they are shown in
 256 Figure 8. In order to simplify goal retrieval, we split the head of the alternatives from the
 257 tail, so that it can be immediately matched in the inductive definition. Note that an empty
 258 list of alternatives represents, by definition, a failing elpi state. If the goal list is empty
 259 (STOPE), then we have, by definition, a success, and the input solution together with the
 260 list of alternatives is returned. If the goal list starts with a cut (CUTE), then the current
 261 alternatives are erased in favour of the cut alternatives, and a recursive call is made on the
 262 remaining goal list.

263 Finally, we must consider the case in which the goal list starts with a call. The call
 264 can either fail (FAILE) or succeed (CALLE). We distinguish the two cases by looking if the
 265 backchaining operation returns zero or more rules. We have wrapped this task in the *stepE*
 266 procedure, which also updates the goal and cut-alternative list. The fail case, is relatively
 267 easy: the first goal does not succeed, we need to take the head of the alternatives, and make
 268 it the new list of goals to be explored.

269 The case in which backchaining produces a non empty list, the *save_alts* routine is in
 270 cahrg of: taking the list of premises and add to each atom the the list of alternatives *a* as
 271 their new cut-alternatives, then it append the list of goals *gl* to each of these new lists.

Definition $\text{stepE } v_0 \ t \ s \ a \ \text{gl} :=$
 $\text{let } (v_1, \text{rs}) := \text{bc } p \ v_0 \ t \ s \ \text{in}$
 $\text{let } \text{rs_ca} := \text{save_alts } a \ \text{gl} \ (\text{r2a } \text{rs}) \ \text{in}$
 $(v_1, \text{rs_ca}).$

Inductive $\text{runE} : \mathcal{F}_v \rightarrow \Sigma \rightarrow \text{goals} \rightarrow \text{alts} \rightarrow \Sigma \rightarrow \text{alts} \rightarrow \text{Prop} :=$ (10)

$$\frac{}{\text{runE } v_0 \ s_0 \ [::] \ a \ s_0 \ a} \text{STOPE}$$

$$\frac{\text{runE } v_0 \ s_0 \ \text{gl} \ \text{ca} \ s_1 \ r}{\text{runE } v_0 \ s_0 \ [:: (\text{cut}, \text{ca}) \ \& \ \text{gl}] \ a \ s_1 \ r} \text{CUTE}$$

$$\frac{\text{stepE } v_0 \ t \ s_0 \ \text{al} \ \text{gl} = (v_1, [:: b \ \& \ \text{bs}]) \quad \text{runE } v_1 \ b.1 \ b.2 \ (\text{bs}++\text{al}) \ s_1 \ r}{\text{runE } v_0 \ s_0 \ [:: (\text{call } t, \text{ca}) \ \& \ \text{gl}] \ \text{al} \ s_1 \ r} \text{CALLE}$$

$$\frac{\text{stepE } v_0 \ t \ s_0 \ \text{al} \ \text{gl} = (v_1, [::]) \quad \text{runE } v_1 \ s_1 \ a \ \text{al} \ s_2 \ r}{\text{runE } v_0 \ s_0 \ [:: (\text{call } t, \text{ca}) \ \& \ \text{gl}] \ [:: (s_1, a) \ \& \ \text{al}] \ s_2 \ r} \text{FAILE}$$

■ **Figure 8** Rule system for runE

6 Semantic equivalence

The equivalence between the two semantics is possible under two conditions: we need to work with “valid tree”, i.e. all of those trees that can be generated from a call. Secondly, we need to translate trees into elpi states. In the next to subsection we propose the two functions *valid_state* and *t2l*.

6.1 Valid trees

The inductive tree allows one to generate a large number of trees, some of which are not valid, in the sense that they cannot be produced starting from a given query. The class of valid trees is characterized by the function shown in Figure 9a.

While all base cases of tree are considered valid, we need to analyze carefully the cases for the *Or* and *And* constructors.

For the *Or* constructor, we distinguish two cases depending on whether the left-hand side (lhs) exists. If it does not exist, then the right-hand side (rhs) must be a valid tree. Otherwise, the lhs must itself be a valid tree, and the rhs is either the *KO* tree, since it may have been removed by the evaluation of a superficial cut in the lhs, or it has not yet been explored. In the latter case, it is a **base_or** tree, namely the right-skewed tree formed by a disjunction of conjunctions that is generated after a successful backchain to a call.

For the *And* constructor, the lhs is required to be a valid tree. The shape of the rhs depends on whether the lhs is a success. If the lhs is not successful, then the rhs has never been explored: the procedures *step* and *next_alt* modify the rhs only when the lhs succeeds. In this case, the lhs must be the right-skewed tree containing conjunctions of premises atoms, the reset point B_0 ensure what shape the rhs should have. If the lhs is a success tree, then the rhs can be modified by *step* and *next_alt*, therefore it must be a valid tree.

```

Fixpoint t2l A s0 bt :=
  match A with
  | OK           ⇒ [:: (s0, [::])]
  | KO           ⇒ [::]
  | TA a         ⇒ [:: (s0, [:: (a, [::]) ])]
  | Or None s1 B ⇒ add_ca_deep bt (t2l B s1 [::])
  | Or (Some A) s1 B ⇒
    let lB := t2l B s1 [::] in
    let lA := t2l A s0 lB in
    add_ca_deep bt (lA ++ lB)
  | And A B0 B ⇒
    let lA := t2l A s0 bt in
    let lB0 := a2g B0 in
    let lA := add_deep bt lB0 lA in
    if lA is [:: (s0, gs) & al] then
      let al := map (catr lB0) al in
      let lB := t2l B s0 (al ++ bt) in
      map (catl gs) lB ++ al
    else [::]
  end.
end.

Fixpoint valid_tree A :=
  match A with
  | TA _ | OK | KO ⇒ true
  | Or None _ B ⇒ valid_tree B
  | Or (Some A) _ B ⇒ valid_tree A &&
    ((B == KO) || B.base_or B)
  | And A B0 B ⇒ valid_tree A &&
    if success A then valid_tree B
    else B == big_and B0
  end.

```

(a) Valid tree

(b) Tree to list

■ Figure 9 Valid tree and Tree to list

6.2 From trees to lists

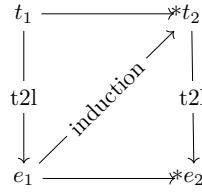
295

296 The translation of a tree to a list is shown in Figure 9b. It takes the tree to be translated, a
 297 substitution (called s), a list of alternatives, called bt . The substitution s tells what is the
 298 substitution of the alternative we are building and is updated when going to the rhs of the
 299 *Or* constructor. The bt list represents the future alternatives list that have the same depth
 300 of the current tree root. They are useful to know if they should be added to the current goal
 301 as cut alternatives.

302 An *OK* node represent a success in a tree, then the translation of this tree returns a
 303 singleton list with the couple $(s, [::])$: there is one alternative with 0 goals, i.e. a success in
 304 elpi by STÖPE. The *KO* node represent a failure, and, therefore 0 alternatives. An atom is
 305 translated into a singleton with the atom as first goal and the empty cut-alternative list: we
 306 are not adding bt since they are the alternatives for the right-sibling, not the right-uncles,
 307 this means that if we have a is a cut, then it erases the alternatives bt .

308 In a disjunction, we are scendendo di un livello the distance from our backtracking list.
 309 This means that bt becomes the list of right-uncles of the two branches of the *Or* constructor.
 310 The compilation of the rhs is done independently of if the lhs exists: we transform the rhs
 311 into an elpi state and passing the empty list of alternatives, since the rhs as no right-siblings.
 312 Then, thanks to `add_ca_deep`, we recursively add bt to every cut-alternative appearing in
 313 the translated goals. If the lhs exists, we translate it and pass the translation of the rhs as
 314 the list of right-siblings. Then we prepend the translated lhs to the translated rhs and add
 315 bt with `add_ca_deep`.

316 We compile the *And* case by translating its lhs to the list lA and reset point to the list
 317 $lB0$. Then, thanks to `add_deep`, we recursively append $lB0$ to the alternatives born in lA ,
 318 i.e. we leave unchanged the pointers to bt , if any, in lA . We finally need to treat cases
 319 whenever the list lA is empty or not. In the former case the empty list is returned, otherwise,



■ **Figure 10** Induction scheme for Lemma 6

we have a list $[(s_0, gs) \& al]$. By the semantics of the *And* in the *runT* interpreter, the rhs of the *And* represent the sequent of goals to be executed after the first alternative in the lhs, it corresponds to the list of goals *gs*. The reset point *B0* is to be appended to the next alternatives in the lhs, that is the queue of *lA*, that is *al*.

alternative in tree? rephrase

We note therefore that the compilation of *lB* is done by passing the alternatives *al* + *bt* since the alternatives in *al* are to be...

6.3 Equivalence theorems

We have now all the ingredients that ...

```

Lemma tree_to_elpi:  $\forall u p s_0 t s_2 t'$ ,
  let  $v_0 := \text{vars\_tree } t \setminus \text{vars\_sigma } s_0$  in
  valid\_tree  $t \rightarrow$ 
  runT  $u p v_0 s_0 t s_2 t' \rightarrow$ 
   $\exists s_1 g a,$ 
  let  $na := t2l \text{ (odflt K0 } t') s_0 [::]$  in
   $t2l \text{ } t s_0 [::] = (s_1, g) :: a \wedge$ 
  runE  $u p v_0 s_1 g a s_2 na.$ 

```

(5)

```

Lemma elpi_to_tree:  $\forall u p v_0 s_1 g s_2 a na,$ 
  runE  $u p v_0 s_1 g a s_2 na \rightarrow$ 
   $\forall s_0 t, \text{valid\_tree } t \rightarrow t2l \text{ } t s_0 [::] = (s_1, g) :: a \rightarrow$ 
   $\exists t', \text{runT } u p v_0 s_0 t s_2 t' \wedge t2l \text{ (odflt K0 } t') s_0 [::] = na.$ 

```

(6)

The proof of Lemma 6 is based on the idea explained in [2, Section 3.3]. An ideal statement for this lemma would be: given a function *l2t* transforming an elpi state to a tree, we would have that the execution of an elpi state *e* is the same as executing *runT* on the tree resulting from *l2t(e)*. However, it is difficult to retrieve the structure of an elpi state and create a tree from it. This is because, in an elpi state, we have no clear information about the scope of an atom inside the list and, therefore, no evident clue about where this atom should be placed in the tree.

Our theorem states that, starting from a valid tree *t* which translates to a list of alternatives $(\sigma_1, g) :: a$. If we run in elpi the list of alternatives, then the execution of the tree *t* returns the same result as the execution in elpi. The proof is performed by induction on the derivations of the elpi execution. We have 4 derivations.

We have 4 cases to analyse:

7 Case study: determinacy analysis

we mechanize the first order part of xxx.

snippet det, main thm, invariant det tree (valid tree prev section?)

proof induction on exec, step/next alt preserving invariant proved by induction on the tree.

with list semantics cut and next alt requires to express a link between the ca or next alts and the current goal, which is non trivial without an intermediate data structure like the tree

8 Related work

prolog semantics, King lost
yves for the proof technique

9 Conclusion

References

- 1 Hassan Ait-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:10.7551/mitpress/7160.001.0001.
- 2 Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: <https://inria.hal.science/inria-00073199v1>.
- 3 Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.
- 4 Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.
- 5 Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPICs*, pages 34:1–34:21, 2020. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPICs.FSCD.2020.34>, doi:10.4230/LIPICs.FSCD.2020.34.
- 6 Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:10.1016/0743-1066(88)90007-6.
- 7 Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λProlog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: <https://inria.hal.science/hal-01176856v1>, doi:10.1007/978-3-662-48899-7_32.
- 8 Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: <https://inria.hal.science/hal-04467855>.
- 9 Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. doi:10.1145/3678232.3678233.
- 10 Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In *Practical Aspects of Declarative Languages: 28th International Symposium, PADL 2026, Rennes, France, January 12–13, 2026, Proceedings*, pages 77–95, Berlin, Heidelberg, 2026. Springer-Verlag. doi:10.1007/978-3-032-15981-6_5.
- 11 Benjamin Grégoire, Jean-Christophe L  chenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023. doi:10.1145/3573105.3575683.
- 12 Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. In *Mathematical Structures in Computer*

- 391 *Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. doi:10.1017/
392 S0960129518000427.
- 393 13 Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least
394 Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors,
395 *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leib-*
396 *niz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany,
397 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: [https://drops.dagstuhl.](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27)
398 [de/entities/document/10.4230/LIPIcs.ITP.2025.27](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27), doi:10.4230/LIPIcs.ITP.2025.27.
- 399 14 Dale Miller. A logic programming language with lambda-abstraction, function variables, and
400 simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.
- 401 15 Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge
402 University Press, 2012.
- 403 16 Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris
404 Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors,
405 *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer
406 Berlin Heidelberg.
- 407 17 Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λ Prolog
408 dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January
409 2018. URL: <https://inria.hal.science/hal-01637063>.
- 410 18 Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of
411 *LIPIcs*, pages 29:1–29:18, September 2019. URL: <https://inria.hal.science/hal-01897468>,
412 doi:10.4230/LIPIcs.CVIT.2016.23.
- 413 19 Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi.
414 Master’s thesis, Radboud University Nijmegen, 2024. doi:10.5281/zenodo.12568604.
- 415 20 David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309,
416 SRI International, Artificial Intelligence Center, Computer Science and Technology Division,
417 Menlo Park, CA, USA, October 1983. URL: [https://www.sri.com/wp-content/uploads/](https://www.sri.com/wp-content/uploads/2021/12/641.pdf)
418 [2021/12/641.pdf](https://www.sri.com/wp-content/uploads/2021/12/641.pdf).